

CS4423 - Networks

Prof. Götz Pfeiffer

School of Mathematics, Statistics and Applied Mathematics

NUI Galway

5. Power Laws and Scale-Free Graphs

Lecture 14: Directed Graphs and The Structure of the World Wide Web

Happy 32nd Birthday, [WWW \(https://webfoundation.org/2021/03/web-birthday-32/\)](https://webfoundation.org/2021/03/web-birthday-32/)!

So far, the networks that have been discussed most of the time consisted of people or organizations, connected by links representing opportunities for interactions. The World Wide Web is an example of a network of a different kind, a so-called **information network**.

Information Networks

Information networks connect pieces of information, like documents, or parts of documents, through links that represent references of some kind. Such links, in contrast to social relationships which are typically symmetric, only point in one direction. The underlying graph of an information network thus is a **directed graph**.

Information networks have existed before the internet. Some prominent examples include:

- **Academic Publications.** In the scientific literature it is customary to give credit to sources that have been used in the form of references to those publications that contain those sources. This practice creates a network whose nodes are the publications, and whose links represent the references, pointing from the citing article back to the cited article. A large part of this network in the mathematical literature is captured on [MathSciNet \(http://www.ams.org/mathscinet\)](http://www.ams.org/mathscinet).
- **Mathematical Proofs.** In mathematics, the proof of a particular theorem usually relies on theorems that have already been proved. Citing a theorem in a proof thus creates a link from the theorem being proved back to the theorem being used, in a network of mathematical theorems. In a similar way, a complex computer program, consisting of several subroutines, can be regarded as a network of subroutines, pointing to each other through links that arise from one subroutine calling another.
- **Technical Documentation.** The documentation of complex systems, such as computer software, typically consists of a collection of articles (manual pages), each describing one aspect of the system, frequently using cross-references to each other. Here the network consists of the manual pages, and the links represent those cross references. In a similar way, an encyclopedia (or a dictionary) organizes its content as a sequence of articles, sorted alphabetically, with supporting cross-references.

Hypertext

The **World Wide Web** arose out of the desire to make technical documentation more easily accessible by using the physical infrastructure of the rapidly growing internet. It was conceived by [Tim Berners-Lee](https://en.wikipedia.org/wiki/Tim_Berners-Lee) (https://en.wikipedia.org/wiki/Tim_Berners-Lee) around 1990 as information management system at [CERN](http://info.cern.ch/hypertext/WWW/TheProject.html) (<http://info.cern.ch/hypertext/WWW/TheProject.html>).

In this system, documents are **web pages**, that anyone can create and store in a publicly accessible place on their computer. Moreover, it supplies a **web browser**, a piece of software that can retrieve the web pages from those public spaces, allowing others to easily access those documents.

Web pages contain **hypertext**, that is a mixture of plain text and **hyperlinks**. Here, a hyperlink (or just link) is a reference to another document that the reader can follow by simply clicking on it. Hyperlinks have a **source** (the document they are contained in) and a **target** (the document they reference). This creates a network of documents as nodes and hyperlinks as **directed edges** between them.

There are many alternative ways to organize information: alphabetically (like the telephone book), hierarchical (in folders like the files on a computer), ... Certainly, the physical constraints of the environment (like the fact that books need to be stored on shelves, that pages in a book come in order) have an influence on how well a particular solution works.

Hypertext makes logical relationships within and between texts (which traditionally are only implicit) explicit first class objects. The browser makes links immediately actionable.

Hypertext originates from the works of the visionaries [Vannevar Bush](https://en.wikipedia.org/wiki/Vannevar_Bush) (https://en.wikipedia.org/wiki/Vannevar_Bush) (the Memex, 1945) and [Ted Nelson](https://en.wikipedia.org/wiki/Ted_Nelson) (https://en.wikipedia.org/wiki/Ted_Nelson) (Xanadu Project, 1965).

It will be useful to distinguish between **navigational links** (providing access to related pages) and **transactional links** (which exist more for a side effect, like ordering a book, or sending an email, than for the sake of leading to the next page). The distinction is not always clear, but transactional links are the kind that is of little interest for search engines. It's the navigational links that form the edges of the directed graph that turns the Web into an information network.

As with undirected graphs, an interesting question in directed graphs is: which nodes can be reached from a given node?

Reachability in Directed Graphs

Recall that a **directed graph** is a pair $G = (X, E)$ with **vertex set** X and **edge set** $E \subseteq X^2 = X \times X$. For an edge $(x, y) \in E$ we sometimes write $x \rightarrow y$.

A **path** in a directed graph $G = (X, E)$ is a sequence of nodes $(x_0, x_1, \dots, x_l)$ with $x_{i-1} \rightarrow x_i$ for $i = 1, \dots, l$. The number l is called the **length** of the path. We write $x \rightsquigarrow y$ if there exists a path (possibly of length 0) from x to y in G .

A directed graph G is **weakly connected** if, when considered as undirected graph, it is connected. The **weakly connected components** (WCCs) of G are its connected components, when considered as undirected graph.

A directed graph G is **strongly connected** if, for each pair of vertices $x, y \in X$, there is a path from x to y in G , i.e., if $x \rightsquigarrow y$.

A **strongly connected component (SCC)** of a directed graph G is a subset C of X which is (i) strongly connected, and (ii) not part of a larger strongly connected subset of X .

In general, a directed graph is a collection of WCCs. Each WCC in turn is a collection of SCCs.

When a directed graph G is regarded as a **relation** on the set X , strongly connected components can be described as the **equivalence classes** of an equivalence relation that is obtained as follows.

First note that the relation $x \rightsquigarrow y$ is the reflexive and transitive closure of the edge relation $x \rightarrow y$. Thus, by construction it is reflexive and transitive. It might not be anti-symmetric, though, meaning that there might be vertices x and y with $x \rightsquigarrow y$ and $y \rightsquigarrow x$.

However, the new relation $x \equiv y$, defined as $x \rightsquigarrow y$ and $y \rightsquigarrow x$ is an equivalence relation (why?) and its equivalence classes are the strongly connected components of G . Denote the class of $x \in X$ by $[x]$.

Moreover, there is a partial order relation $\leq$ (a relation which is reflexive, transitive and anti-symmetric) on the set of equivalence classes, $[x] \leq [y]$ if $x \rightsquigarrow y$.

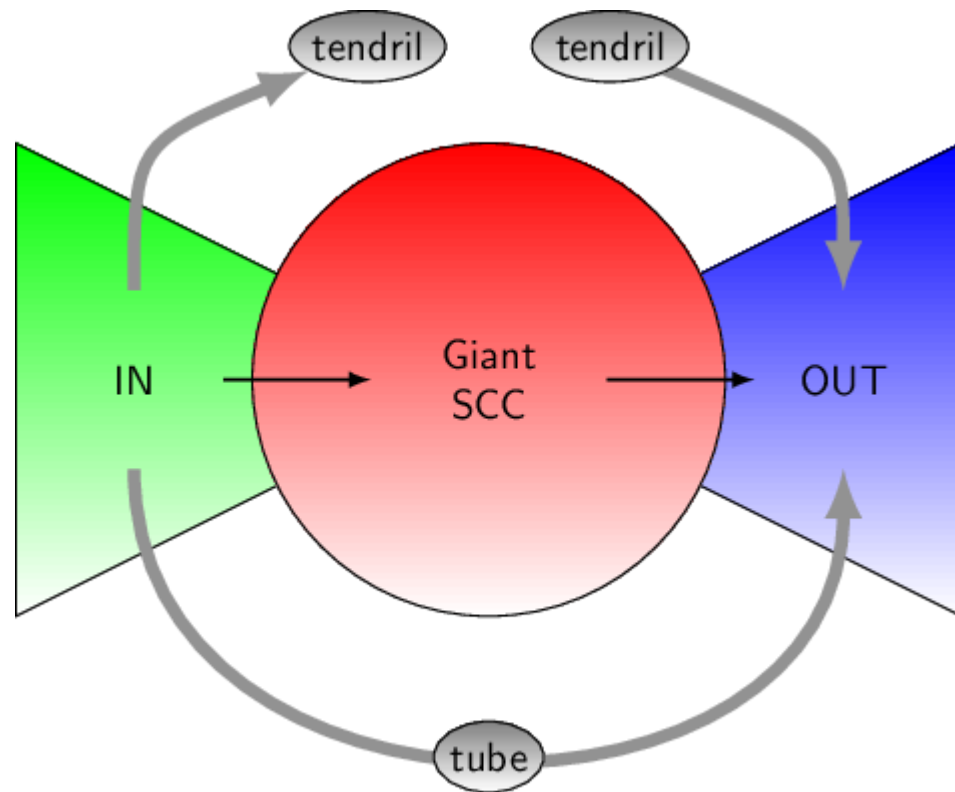
The Bow-Tie Structure of the WWW

Like the giant component in a simple graph, a directed graph with sufficiently many edges has a **giant SCC**.

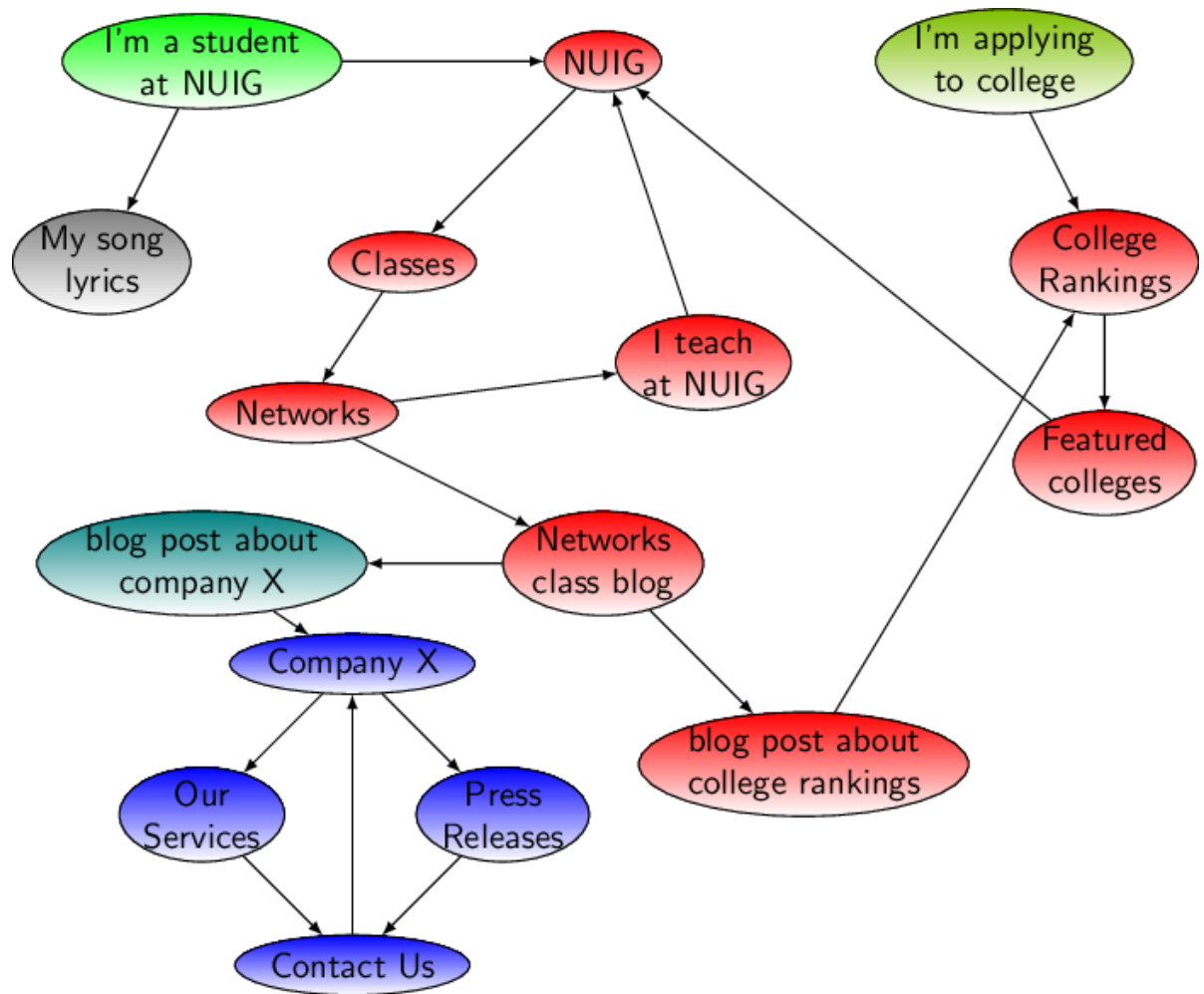
The remainder of the graph consists of four more sets of components of nodes.

1. IN: upstream components, the set of all components C with $C < \text{SCC}$.
2. OUT: downstream components, the set of all components C with $C > \text{SCC}$.
3. tendrils: the set of all components C with either $C > \text{IN}$ and $C \not\leq \text{OUT}$ or $C < \text{OUT}$ and $C \not\leq \text{IN}$;
and tubes: components C with $C > \text{IN}$, $C < \text{OUT}$ but $C \not\leq \text{SCC}$.
4. disconnected components.

Thus, in any directed graph with a distinguished SCC, the containing WCC necessarily has the following global bow-tie structure:



For example,



Research on available data on the Web in 1999 has confirmed this bow tie structure for the World Wide Web, with a large Giant SCC covering less than $\frac{1}{3}$ of the vertex set, and the three parts IN, OUT and the tendrils and tubes roughly of the same size. One can assume that this proportion has not changed much over time, although the advent of social media has brought many new types of links and documents to the Web.

Computing Bow-Tie Components

```
In [1]: import networkx as nx
import matplotlib.pyplot as plt
opts = { "with_labels" : True, "node_color" : 'y' }
```

Example. Let's start with a reasonably large random **directed graph**, using the Erdős-Rényi $G(n, m)$ model:

```
In [2]: n, m = 100, 120
G = nx.gnm_random_graph(n, m, directed=True)
```

Weakly Connected Components

The weakly connected components of a directed graph G can be determined by BFS, as before, counting as "neighbors" of a node x **both** its "successors" and its predecessors in the graph.

A single component, the weakly connected component of node x , is found as follows.

```
In [3]: def weak_component(G, x):
        nodes = {x}
        queue = [x]
        for y in queue:
            G.nodes[y]["seen"] = True
            for z in set(G.successors(y)) | set(G.predecessors(y)):
                if z not in nodes:
                    nodes.add(z)
                    queue.append(z)
        return nodes
```

The list of all weakly connected components is computed by a loop over all the nodes of G , collecting the components of "unseen" nodes in a list. The final result is sorted by length, descendingly, before it is returned.

```
In [4]: def weak_components(G):

        # initialize
        wccs = []

        # find each node's wcc
        for x in G:
            if not G.nodes[x].get("seen"):
                wccs.append(weak_component(G, x))

        # clean up after yourself
        for x in G:
            del G.nodes[x]["seen"]

        # return sorted list of wccs
        return sorted(wccs, key=len, reverse=True)
```

```
In [5]: wccs = weak_components(G)
        len(wccs)
```

Out[5]: 11

```
In [6]: [len(c) for c in wccs]
```

Out[6]: [87, 3, 2, 1, 1, 1, 1, 1, 1, 1]

Strongly Connected Components

Strongly connected components are efficiently found by DFS. [Tarjan's Algorithm](https://en.wikipedia.org/wiki/Tarjan%27s_strongly_connected_components_algorithm) (https://en.wikipedia.org/wiki/Tarjan%27s_strongly_connected_components_algorithm) cleverly uses recursion and an additional stack for this.

The following function finds strongly connected components in a recursive fashion.

```
In [7]: def connect(G, stack, sccs, idx, x):
        G.nodes[x]["low"] = G.nodes[x]["idx"] = idx
        idx += 1
        stack.append(x)
        G.nodes[x]["stacked"] = True
        for y in G[x]:
            if "idx" not in G.nodes[y]:
                idx = connect(G, stack, sccs, idx, y)
                G.nodes[x]["low"] = min(G.nodes[x]["low"], G.nodes[y]["low"])
            elif G.nodes[y]["stacked"]:
                G.nodes[x]["low"] = min(G.nodes[x]["low"], G.nodes[y]["idx"])

        if G.nodes[x]["low"] == G.nodes[x]["idx"]:
            scc = []
            while True:
                y = stack.pop()
                G.nodes[y]["stacked"] = False
                scc.append(y)
                if y == x:
                    break
            sccs.append(scc)

        return idx
```

Similar as in the case of the weakly connected components, the overall algorithm uses a loop over all the nodes of G to find all strongly connected components.

```
In [8]: def strong_components(G):

    # initialize
    idx = 0
    stack = []
    sccs = []

    # find each node's scc
    for x in G:
        if "idx" not in G.nodes[x]:
            idx = connect(G, stack, sccs, idx, x)

    # clean up after yourself
    for x in G:
        del G.nodes[x]["idx"]
        del G.nodes[x]["low"]
        del G.nodes[x]["stacked"]

    # return sorted list of sccs
    return sorted(sccs, key = len, reverse=True)
```

```
In [9]: sccs = strong_components(G)
        [len(c) for c in sccs[:10]]
```

```
Out[9]: [24, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

As the resulting list of components is sorted by length, in descending order, `sccs[0]` is the **Giant SCC**.

The OUT Components

The components reachable from the Giant can be found by BFS applied to any vertex in the Giant. (Why?) So here is a node representating the Giant:

```
In [10]: rep = min(sccs[0])
        rep
```

```
Out[10]: 1
```

This variant of BFS considers the `successors` of a node `x` as its neighbors.

```
In [11]: def outreach(G, x):
        """find all nodes that can be reached
        from node x in the directed graph G"""
        nodes = {x}
        queue = [x]
        for y in queue:
            for z in G.successors(y):
                if z not in nodes:
                    nodes.add(z)
                    queue.append(z)
        return nodes
```

The resulting list of nodes consists of the Giant and the OUT components.

```
In [12]: out = outreach(G, rep)
        len(out)
```

Out[12]: 49

The components reachable from the Giant can be found by BFS applied to any vertex in the Giant, following the arrows in reverse. (Why?)

This variant of BFS considers the predecessors of a node x as its neighbors.

```
In [14]: def innreach(G, x):
        """find all nodes that can reach
        node x in the directed graph G"""
        nodes = {x}
        queue = [x]
        for y in queue:
            for z in G.predecessors(y):
                if z not in nodes:
                    nodes.add(z)
                    queue.append(z)
        return nodes
```

The resulting list of nodes consists of the Giant and the IN components.

```
In [15]: inn = innreach(G, rep)
        len(inn)
```

Out[15]: 46

The Giant is the intersection of `inn` and `out`. (Why?)

```
In [16]: giant = inn & out
```

Let's call the union of `inn` and `out` the **core** of the graph G .

```
In [17]: core = inn | out
len(core)
```

```
Out[17]: 71
```

And let's remove the Giant from `inn` and `out` :

```
In [18]: inn1 = inn - out
out1 = out - inn
len(giant), len(inn1), len(out1)
```

```
Out[18]: (24, 22, 25)
```

Tubes and Tendrils

Tubes and tendrils can be found by BFS, after reversing all arrows that have both their endpoints in `core` . (Why?)

So, let's make a copy of `G` and modify it accordingly.

```
In [19]: H = G.copy()
flip = [e for e in H.edges if set(e).issubset(core)]
len(flip)
```

```
Out[19]: 92
```

```
In [20]: for e in flip:
          H.remove_edge(*e)
          H.add_edge(e[1], e[0])
```

Now `outreach` applied to the node `rep` in the new graph `H` finds a set of nodes `out2` outside the core. Similarly, `innreach` finds a set of nodes `inn2` outside the core.

The intersection of `inn2` and `out2` is the set of nodes in tubes. (Why?) Removing the tube nodes from both `inn2` and `out2` leaves two sets of nodes in tendrils.

```
In [21]: out2 = outreach(H, rep) - core
inn2 = innreach(H, rep) - core
tube2 = inn2 & out2
core2 = inn2 | out2
inn2 = inn2 - tube2
out2 = out2 - tube2
len(tube2), len(inn2), len(out2), len(core2)
```

```
Out[21]: (4, 6, 3, 13)
```

If `tube2` now is the empty set this example is not very interesting. Go back and start over. (If necessary, change the values of `n` and `m` .)

In [22]: tube2

Out[22]: {2, 23, 55, 73}

Drawing the Bow-Tie

The subgraph of G consisting of the core, the tubes and the tendrils can be drawn in a way the exhibits the bow-tie structure, by using different colors and different regions in the diagram for the different kinds of components.

First we define this subgraph.

In [23]: `S = nx.subgraph(G, core | core2)`

Next we compute positions for the different groups of nodes individually, and horizontally shift these groups into non-overlapping regions.

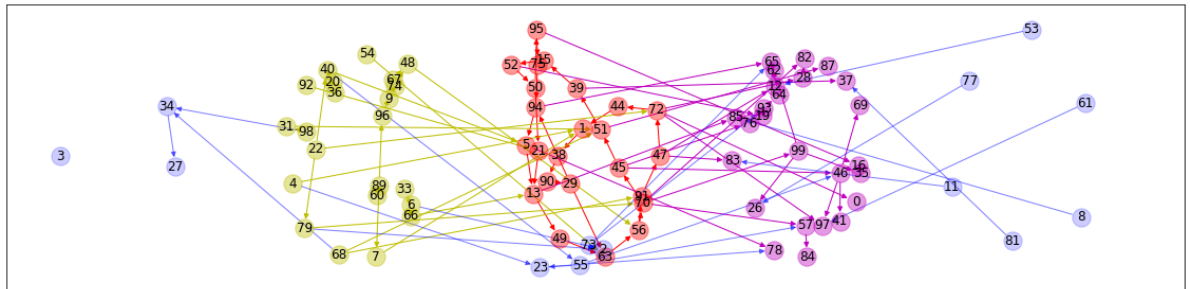
```
In [24]: pos = nx.spring_layout(nx.subgraph(G, giant))
pos_inn1 = nx.spring_layout(nx.subgraph(G, inn1))
for v in pos_inn1.values():
    v[0] = 2/3 * v[0] - 2
pos_inn2 = nx.spring_layout(nx.subgraph(G, inn2))
for v in pos_inn2.values():
    v[0] = 2/3 * v[0] + 4
pos_out1 = nx.spring_layout(nx.subgraph(G, out1))
for v in pos_out1.values():
    v[0] = 2/3 * v[0] + 2
pos_out2 = nx.spring_layout(nx.subgraph(G, out2))
for v in pos_out2.values():
    v[0] = 2/3 * v[0] - 4
pos_tube = nx.spring_layout(nx.subgraph(G, tube2))
for v in pos_tube.values():
    #v[0] *= 2
    v[1] = v[1]/10 - 1
pos.update({ **pos_inn1, **pos_inn2, **pos_out1, **pos_out2, **pos_tube
```

Finally, using the precalculated positions, the different groups of nodes, and (certain) edges between them are drawn individually, with different colors to tell them apart.

```

In [25]: plt.figure(figsize=(20,5))
nx.draw_networkx_nodes(S, pos, nodelist=giant, node_color='r', alpha=0.4)
nx.draw_networkx_nodes(S, pos, nodelist=inn1, node_color='y', alpha=0.4)
nx.draw_networkx_nodes(S, pos, nodelist=out1, node_color='m', alpha=0.4)
nx.draw_networkx_nodes(S, pos, nodelist=core2, node_color='b', alpha=0.2)
edges = [e for e in S.edges if set(e) <= giant]
nx.draw_networkx_edges(S, pos, edgelist=edges, edge_color = 'r')
edges = [e for e in S.edges if set(e) <= out1]
nx.draw_networkx_edges(S, pos, edgelist=edges, edge_color = 'm')
edges = [e for e in S.edges if e[0] in giant and e[1] in out1]
nx.draw_networkx_edges(S, pos, edgelist=edges, edge_color = 'm')
edges = [e for e in S.edges if set(e) <= inn1]
nx.draw_networkx_edges(S, pos, edgelist=edges, edge_color = 'y')
edges = [e for e in S.edges if e[0] in inn1 and e[1] in giant]
nx.draw_networkx_edges(S, pos, edgelist=edges, edge_color = 'y')
edges = [e for e in S.edges if e[0] in inn1 and e[1] in out1]
#nx.draw_networkx_edges(S, pos, edgelist=edges, edge_color = 'k')
edges = [e for e in S.edges if set(e).issubset(tube2)
        or set(e).issubset(inn2) or set(e).issubset(out2)]
edges += [e for e in S.edges if e[0] in inn1 and e[1] in tube2]
edges += [e for e in S.edges if e[0] in tube2 and e[1] in out1]
edges += [e for e in S.edges if e[0] in inn2 and e[1] in out1]
edges += [e for e in S.edges if e[0] in inn1 and e[1] in out2]
nx.draw_networkx_edges(S, pos, edgelist=edges, edge_color = 'b', alpha=0.2)
nx.draw_networkx_labels(S, pos);

```



```

In [27]: print(giant)

```

```

{1, 5, 13, 15, 21, 29, 38, 39, 44, 45, 47, 49, 50, 51, 52, 56, 63, 70,
72, 75, 90, 91, 94, 95}

```

```

In [ ]:

```

Code Corner

python

- setX & setY, setX | setY, setX - setY: [\[doc\]](https://docs.python.org/2/library/stdtypes.html#set)
(<https://docs.python.org/2/library/stdtypes.html#set>) set operations, intersection, union, difference
- x in setY, x not in setY: set membership

- `setX.issubset(setY)` , `setX <= setY` , `setX < setY` : subset relationship

networkx

- `draw_networkx_nodes` : [\[doc\]](#)
(https://networkx.github.io/documentation/stable/reference/generated/networkx.drawing.nx_pydraw_networkx_nodes.html)
- `draw_networkx_edges` : [\[doc\]](#)
(https://networkx.github.io/documentation/stable/reference/generated/networkx.drawing.nx_pydraw_networkx_edges.html)
- `draw_networkx_labels` : [\[doc\]](#)
(https://networkx.github.io/documentation/stable/reference/generated/networkx.drawing.nx_pydraw_networkx_labels.html)



Exercises

1. Use the "idx" information as collected by Tarjan's algorithm for the horizontal positioning of the nodes when drawing a graph.
2. Explain why this might be useful.
3. Read the paper by [Timár et al. \(https://arxiv.org/abs/1607.00691\)](#) on how the simple bow-tie structure discussed here can be iterated.
4. Implement the algorithms from the paper in (3.) to compute such an iterated bow-tie decomposition of a directed network.

In []: